Systems Administration (ASIST)

Practical Theoretical Class 05

Servers administration.
Remote access. Software management.
Scripts. Task scheduler.
Centralized Storage - SAN - iSCSI.

Server management – remote access

Once server IP addresses are set, and preferably registered in DNS, the administrator can choose to use appropriate application protocols to access and administer the server and to discontinue use of the console.

Linux servers: without graphical interface, the most appropriate form of access is using the Secure Shell (**SSH**). Simply install an SSH server such as ***OpenSSH Server***. SSH is a terminal emulation made over a *Transport Layer Security* (**TLS**) protected TCP connection. In this type of use the server is not authenticated by public key certificate. When the user connects to a server, it provides his RSA key, if unknown, the user is alerted.

Windows servers: the simplest form of remote access is through the Remote Desktop Protocol (**RDP**). Simply activate through *Server Manager* and use an appropriate client (*Remote Desktop Connection* - **RDC**). Another form of remote server management is through *Server Manager*. This application allows management not only of the local server where it runs, but also of other servers across the network.

Software management

After the initial installation of the server it becomes essential to have tools to expeditiously update and install new necessary software.

Linux: For most distributions there is a set of repositories on the Internet through which updates are provided and access to various software packages to be installed. The application/service used for this purpose depends on the distribution, in the case of the Debian family (which includes Ubuntu), package management system is controlled by the **apt** (*Advanced Package Tool*) command.

Windows Server: Most essential software is available locally after installation, can be managed through ***Server Manager*** by adding/removing roles and features. *Role* is a set of software packages that allows to perform a particular function. *Features* are software packages that doesn't take part of a role but provide other functions that might be related to several roles. It is normal the installation of a role require the installation of some features. One of the roles available is *Windows Server Update Services* (**WSUS**) that enables a software update service to be made available to the local systems.

Linux - APT (Advanced Package Tool)

In Debian family distributions the software package manager is **apt.** Update installation can also be automated by installing the **unattended-upgrades** package.

Essential **apt** commands:

Command	Description
apt update	Updates the list of available packages by downloading existing package lists from various repositories.
apt install {PACKAGE-NAME}	Prompts for package installation.
apt remove {PACKAGE-NAME}	Prompts to remove package, configuration files changed by administrator are not removed.
apt purge {PACKAGE-NAME}	Prompts for the removal of the package including the configuration files (valid even if the package has already been removed).
apt dist-upgrade	Request a general update of all installed packages. Might include removal of packages no longer needed.
apt autoremove	Prompts to remove all installed packages that are no longer in use (obsolete packages that have already been replaced by others).

Linux - APT (Advanced Package Tool)

Command	Description
apt list [--installed] [PACKAGE-NAME]	Without options, lists all existing packages. With the --installed option, only installed packages. You can specify a package name or a pattern. Using a pattern makes it simple to look for a package you want. For example, for a list of all existing packages whose name contains the word **libnss**, use the command: **apt list *libnss***
apt search {STRING}	Allows you to search for packages via a STRING; STRING matches will be searched for in the name as well as in the detailed package description.
apt show {PACKAGE-NAME}	Displays all the details about the indicated package.

apt uses **dpkg** (*package manager for Debian*) but not all *dpkg* possibilities are available in *apt*. Frequently, during package installation, installation scripts are executed that prompt the user for the settings to be applied. If it is needed to change these settings after installation, the command to use will be:

dpkg-reconfigure {PACKAGE-NAME}

WSUS(*Windows Server Update Services*)

Like any operating system, on Windows Server we should automate updates, the recommended option is to **automatically download and install updates**, to keep track we can choose to **download updates and ask for confirmation before installing them.**

Installing new applications on a Windows server other than those available through *Server Manager* (*roles and features*) should be done with caution, preferably Microsoft applications whose compatibility is guaranteed for the operating system version concerned.

To download installation packages using Internet Explorer, the administrator must disable Internet Explorer *Enhanced Security Configuration* (**IE ESC**) mode through Server Manager.

WSUS Server role can be enabled on a Windows Server to obtain updates from Microsoft and make them available to local Windows machines by configuring it to obtain updates for the intended operating systems and languages. You can also embed other Microsoft applications like new versions of IE. You can control which updates will apply to different local computer groups and the application time. Local computers are configured through a *Group Policy* to obtain local WSUS server updates.

WDS (*Windows Deployment Services*)

WDS allows you to manage a computer park consisting of a large number of Windows machines. WDS can only be applied in the context of an Active Directory domain, it also requires a DHCP server and workstations where network interfaces support PXE (*Preboot eXecution Environment*).

Allows:

- *Unattended* Installation of the Operating System from a Network *Boot* (PXE). Disk image can be multicasted, installing on many machines simultaneously.
- The operating system is managed on a WDS server resident image named **imageX**. The image can be created on a desktop and copied to the server or fully created on the server.
- On the WDS server you can change the image (**imageX**), namely install software and drivers and automatically apply the changes to all workstations (*deploy*).
- During the installation process on each workstation, operating system installation procedures are performed to adapt it to the hardware.
- During installation the machine name is set correctly and the machine can be automatically entered in the Windows domain.

Scripts

Currently this is a very broad concept. *Scripts* began as batch files, ie text files with a sequence of commands, the batch file can be invoked causing sequential execution of commands (*batch execution*).

Currently all *scripting* languages also include commands that allow you to define variables, decision structures and repetition resembling very real programs. The current concept of *scripting* is strongly associated with the fact that it is executed by an interpreter program and is not a compiled program.

For a *script* (text file) to be executed directly by the operating system without having to explicitly invoke the interpreter program, the operating system must recognize it as such.

Windows: The content type of each file is identified by its name, more exactly its **extension** (3-character set at the end of the name after the dot). For each extension the operating system has defined how to handle the file, in the case of *scripts*, the interpreter program that must execute to process it.

Linux: the content type of each file is identified by the ***magic number*** of its content, the name is completely ignored.

Unix - Magic number

Unix operating systems identify file content type through ***magic numbers***. A ***magic number*** is a set of bytes with fixed values existing at certain fixed positions of the file that allows its contents to be identified.

At Shell, the **file** command can determine what type it is a file system object. After verifying that it is not a folder, symbolic link, or special object, check its contents and check against known ***magic numbers***.

The *magic number* that identifies a script consists of the characters **#!** in the first two positions of a file. This sequence at the beginning of a file is called **shebang**, immediately following it, on the same line of the text file, the interpreter program that should be used to process the content must be identified.

```
#!/bin/bash
--- --
-- -- --
```

```
#!/usr/bin/perl
--- --
-- -- --
```

```
#!/bin/sh
--- --
-- -- --
```

```
#!/bin/csh
--- --
-- -- --
```

```
#!/usr/bin/pyton
--- --
-- -- --
```

Additionally the file must have execute permission in order to be executed directly by the operating system. Note that lines beginning with the **#** character are considered comments by interpreters so the **shebang** line does not interfere with script execution.

```
#!/usr/bin/php
--- --
-- -- --
```

Linux – Shell scripts

These are scripts that are executed by the command line interpreters commonly called Shell, especially Bourne Shell (sh/bash) and C Shell (csh/tcsh), among others.

If a file starts with shebang but does not identify the interpreter program on that line, the default Shell will be used, usually **bash** (although it depends on the distribution).

Both the traditional *Bourne Shell* and *C Shell* are available in any Linux installation therefore constitute an important tool that is always available to the administrator to automate tasks, simply having a text editor.

Scripts written in C Shell use a syntax that is very similar to the C programming language, so they are interesting for administrators who usually work with C language.

Other types of languages interpreted like *Perl* and *Python* have much greater potential, with quite extensive libraries, but require administrator familiarity.

The example shown at right, written in C Shell, prints all powers of two less than 1100.

```
#!/bin/csh
echo "Starting"
@ num = 1
while($num<1100)
 echo $num
 @ num = 2 * $num
end
```

Linux – Shell scripts

The following example, written in *bash*, takes a number as an argument on the command line (${1} is the first argument) and prints the factorial value of that number.

```
#!/bin/bash
echo "Factorial Calculator"
if [ -z "${1}" ]
        then
        echo "Must provide argument (positive integer)"
        exit 1
fi
NUM=${1}
FACT=1
while [ ${NUM} -gt 1 ]
        do
        FACT=$((${FACT}*${NUM}))
        NUM=$((${NUM}-1))
done
echo "Factorial of ${1} = ${FACT}"
#### DONE
```

The placement of braces surrounding the names of the referenced variables is not mandatory. Spaces placed in the context of condition tests (**if** and **while**) are fundamental.

Windows scripts – WSH (Windows Script Host)

In Windows operating systems scripts are identified by their extension:

Extension	Script type
.BAT .CMD	Files Processed by Command Line Interpreter (cmd.exe).
.JS .JSE	**JScript** – Interpreted language based on Microsoft Visual J ++ 6.0.
.VBS .VBE	**VBScript** – Interpreted language based on Microsoft Visual Basic 6.0.
.WSF	**Windows Script File** – Uses XML to support different languages in the same script, for example JScript and VBScript..
.PS1	**Windows PowerShell** shell script



Today's Windows systems have a scripting support platform called *Windows Script Host* (**WSH**).

This platform allows compatible interpreted languages, namely JScript and VBScript, to access a wide range of functionality through objects that can be used. Through WSH it is for example possible to use *Windows Management Instrumentation* (**WMI**) and *Active Directory Service Interfaces* (**ADSI**).

WSH (Windows Script Host) - Example

In this JScript example WSH is used to access WMI and get a list of user accounts. For each account, a window with the username and last login date is displayed successively:

```
LoginProfiles = GetObject("winmgmts:").InstancesOf ("Win32_NetworkLoginProfile");
for(e = new Enumerator(LoginProfiles) ; !e.atEnd() ; e.moveNext())
{
   Profile = e.item();
   lastLog="never";
   if(Profile.LastLogon!=null) lastLog=Profile.LastLogon;
   WScript.Echo(Profile.Name + " : " + lastLog);
}
```

The following example in VBScript does the same:

```
Set LoginProfiles = GetObject("winmgmts:").InstancesOf ("Win32_NetworkLoginProfile")
   for each Profile in LoginProfiles
      WScript.Echo Profile.Name
      WScript.Echo Profile.LastLogon
   next
```

WSF (Windows Script File) - Example

A WSF script uses XML and can contain different languages:

```
<job>
<script language="VBScript">
  MsgBox "I am now running VBScript, but next I will run JScript"
</script>
<script language="JScript">
  WSH.echo("Now I am running JScript and I will show the user accounts.");

  LoginProfiles = GetObject("winmgmts:").InstancesOf ("Win32_NetworkLoginProfile");
  num=0;
  for(e = new Enumerator(LoginProfiles) ; !e.atEnd() ; e.moveNext())
   {
   Profile = e.item();
   lastLog="never";
   if(Profile.LastLogon!=null) lastLog=Profile.LastLogon;
   WScript.Echo(Profile.Name + " : " + lastLog);
   num++;
   }
  WScript.Echo("Were presented " + num + " user accounts");
  WScript.quit();
</script>
</job>
```

Windows PowerShell

PowerShell provides an additional set of commands called ***cmdlets***, allowing you to administer all server functionality. It is designed to ease the administration tasks of Windows servers.

The number of cmdlets is immense, the names are oriented by the intended action verb, for example to query the state commands are started by Get-, to modify commands started by Set-.

Firstly, the **Get-help** command can be used to get help, for example **Get-help service** lets you get a list of cmdlets whose name contains **service**. Detailed cmdlet help can then be requested directly, for example **Get-help Start-service**.

PowerShell also has *autocomplete* via TAB key.

Script development is greatly facilitated by the *Integrated Scripting Environment* (**ISE**). It is a graphical mode PowerShell script development tool with direct assistance available, providing information on all available cmdlets.

The scripts (files with extension **.PS1**) are edited in ISE and can be tested in the environment itself. The PowerShell ISE Script Development Environment is Available in Windows Server ***Administrative Tools***.

Windows PowerShell script – examples WMI

```
# WMI query to list all properties and values of the Win32_BIOS class
# http://www.robvanderwoude.com

param( [string]$strComputer = "." )

$colItems = get-wmiobject -class "Win32_BIOS" -namespace "root\CIMV2" -computername $strComputer

foreach ($objItem in $colItems) {
          write-host "Name                            :" $objItem.Name
          write-host "Version                         :" $objItem.Version
          write-host "Manufacturer                    :" $objItem.Manufacturer
          write-host "SMBIOSBIOS Version              :" $objItem.SMBIOSBIOSVersion
          write-host
}

# WMI query to list all properties and values of the Win32_Printer class
# http://www.robvanderwoude.com

$colItems = get-wmiobject -class "Win32_Printer" -namespace "root\CIMV2" -computername $strComputer

foreach ($objItem in $colItems) {
          write-host "Name                            :" $objItem.Name
          write-host "Default                         :" $objItem.Default
          write-host "Network                         :" $objItem.Network
          write-host "Port Name                       :" $objItem.PortName
          write-host "Driver Name                     :" $objItem.DriverName
          write-host "Server Name                     :" $objItem.ServerName
          write-host "Share Name                      :" $objItem.ShareName
          write-host
}
```

Task Scheduler

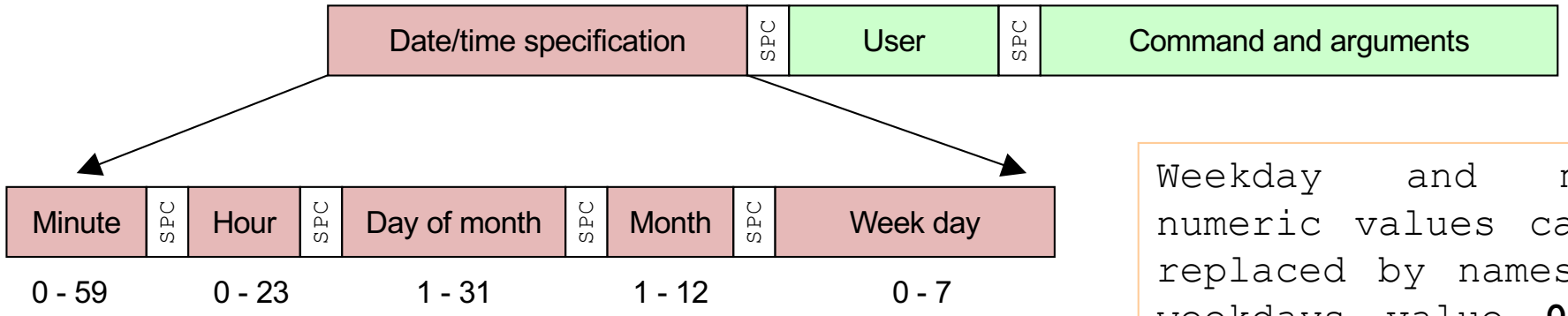
Scheduled task execution is a key server administration tool as it allows the administrator to set *unattended* execution of applications and scripts at appropriate times. A typical example is backup operations.

Windows: ***Task Scheduler*** graphical application allows to schedule tasks. It is possible to specify a wide range of parameters regarding the execution of tasks, from which user account will be used and specific conditions that may condition the execution of the task. In addition to hourly scheduling per second, tasks can also be scheduled to run in other types of situations, such as at machine startup or when the machine is *idle*. It can also be set a task expiration date, the task will not be deleted but will be inactive from that date and will not run. The minimum granularity is 1 minute.

Linux: on Unix systems in general the most commonly used task scheduler is **cron**. It is a service that is running continuously that reads the configuration file **/etc/crontab** every minute and performs the applicable tasks in that minute. Through the settings in this file we can set execution times based on time, minute, day of the month and day of the week. The minimum granularity is therefore also the minute.

Linux – cron service

Scheduled task execution on Linux systems usually uses the cron service. Usually the administrator defines the tasks in the text file **/etc/crontab**. Each line represents a task to be executed as follows:



Date/time values can be specified in ranges or by comma-separated enumeration of values. The ***** symbol represents the enumeration of all possible values for this field. For example for minutes, enumeration of all possible values can be done in the form **0-59** or *****.

Weekday and month numeric values can be replaced by names. On weekdays value **0** and value **7** are two alternatives for specifying **Sunday**.

From minute to minute the cron service reads this file and compares the current date/time with the specifications in the file. For each match found execute the command using the specified user account.

Linux – example /etc/crontab

```
*/5 * * * * root /usr/bin/mrtg /etc/mrtg/mrtg.cfg
5,35 * * * * root /etc/LinuxHealth >/dev/null 2>&1 &
45 2 * 8 6 root /root/make-backup >/dev/null 2>&1 &
```

The enumerations of values can be associated with a ***step***, in the example above, in minutes, ***/5** (equivalent to **0-59/5**) is equivalent to **0,5,10,15,20,25,30,35,40,45,50,55.** In this case (as the other specifications are *****) the task will run every five minutes whenever the date / time value has a corresponding number of minutes.

The **/etc/crontab** file can only be edited by the administrator, however the *CRON* service is also accessible to other users via configuration files normally stored in **/var/spool/cron/*username*.** These must be handled indirectly through the command **crontab -e.**

The format is identical to **/etc/crontab**, but the username (6th field) is omitted as it is implied, a user's tasks defined in this way are always performed using that user's account. User access to the *CRON* service can be controlled by the administrator via the **/etc/cron.allow** and **/etc/cron.deny** files.

Centralized *Storage* – SAN and NAS

Centralized deployment across the network of storage devices can be accomplished through two distinct approaches, *Storage Area Network* (**SAN**) and *Network Attached Storage* (**NAS**).

On the NAS server local file system folders are made available to clients, unlike logical disks the responsibility for managing the file system lies with the NAS server.

Windows systems use the *Server Message Block/Common Internet File System* (**SMB/CIFS**) protocol to share folders. Unix systems use the *Network File System* (**NFS**) protocol to **export folders**.

The terminology is different, but the concept is the same.

SAN (Storage Area Network) - iSCSI

The ***iSCSI*** protocol allows you to create a SAN over any IP network infrastructure, requiring no special hardware. Windows servers and Linux servers can be *storage servers* – **targets** - or *storage clients* – **initiators** - that use logical disks.

iSCSI intervening nodes (***targets*** and ***initiators***) are identified by special names, typically in *iSCSI Qualified Name* (**IQN**) format, as:

iqn.{YYYY-MM}.{inverted-dns-name}:{name}

Where **{YYYY-MM}** represents the year and month the DNS name was created, **{inverted-dns-name}** represents the reverse domain's DNS name and **{name}** is an arbitrary name that ensures that the pool is unique.

For example the node name (in the case of a target usually this name also identifies the logical disk).

IQN example of a target: **iqn.2000-08.pt.ipp.isep.dei:server1-lun0**

The IQNs used by *targets* can be discovered through the *discovery* function as long as their IP address or DNS names are known.

Initiators' access to *targets* is controlled via *username* and *password* using the **CHAP** algorithm.

iSCSI Initiator – Windows

Current Windows systems (including Windows Server) can use target-enabled logical disks, ie operate as *initiator*. To manage this feature should be used the **iSCSI Initiator** application available in the **Administrative Tools**.

The target can be added via the respective IP address or DNS name, you must then connect to the target (*Connect*) by setting the username and password for CHAP authentication (advanced *Connect* options) as defined by the target.

If the target is added to the favorites list, the connection will be made automatically whenever the machine starts.

After binding to the target, we can choose to automatically configure all logical disks (volumes) or just some of those available in the target. the volumes being configured become new discs available as if they were local.

In the case of a newly created empty disk, through **Computer Management** we must bring the disk online and initialize it (create the partition table) after which new partitions can be created to be formatted and assigned drive letters.

For **Computer Management**, it's all about a local physical disk. As such all operations on local disks are available including being converted to ***dynamic***.

iSCSI Target – Windows

Windows Server 2012 can function as an iSCSI storage server (ie iSCSI target). For this purpose the roles **File Server** and **iSCSI Target Server** must be installed (they are in the **File and Storage Services** role set). The **iSNS Server** (*discovery services*) feature will also be installed. Through **Server Manager** it is then possible to create logical disks to be made available by the target.

Only virtual disks are supported, ie disks that are stored as a large file (corresponding to disk size.

When creating the virtual disk we have to define its size, later it can be expanded if necessary but cannot be shrunk.

Virtual disks can be created in many ways:
Fixed Size means that a file with the size corresponding to disk size will be created immediately;
Dynamically Expanding causes the file size to grow as data is placed on the disk;
Differencing allows you to create a new disk based on an existing disk, the existing disk is preserved in its initial state, and changes to it are recorded on the new disk.

Once the virtual disk has been created, the corresponding target can then be set.

iSCSI Target – Linux

In **Ubuntu 18.04** the package that allows you to create an iSCSI target is tgt. After its installation the configuration is done by command **tgtadm.**

Created target names (IQN) must match disks to be made available. For ease of creation a **conf.d** folder is created under **/etc/tgt** where you can create several files that are always included - as can be seen in **/etc/tgt/targets.conf.**

A logical disk can be a physical disk, for example /dev/sdc, it can also be a partition, for example /dev/sdb3, or like Windows Server it can also be a file that acts as a virtual disk. Setting the target also sets authentication credentials through CHAP. Example:

```
Target iqn.2015-10.pt.ipp.isep.dei:storage.lun0
        IncomingUser root PASSWORD
        OutgoingUser
        Lun 0 Path=/home/storage/lun0.disk,Type=fileio
```

In this case the target is a virtual disk stored in the file **/home/storage/lun0.disk**, previously created with the desired size for the logical disk and can initially be filled with any data. Later it will be formatted by the *initiator* that uses it.

iSCSI Initiator – Linux

In **Ubuntu 18.04** the package that lets you use logical disks made available by iSCSI targets is **open-iscsi** (*Open-iSCSI project* http://www.open-iscsi.org/). The first step is to find out the target names by command:

iscsiadm -m discovery -p {SERVER} -t st

Where **{SERVER}** is the IP address or DNS name of the target machine. Once the desired logical disk IQN name is identified, the authentication method (CHAP), username and password must be defined for that logical disk.

```
iscsiadm -m node -T {IQN} –p {SERVER} –o update –n node.session.auth.authmethod –v CHAP
iscsiadm -m node -T {IQN} –p {SERVER} –o update –n node.session.auth.username –v {USERNAME}
iscsiadm -m node -T {IQN} –p {SERVER} –o update –n node.session.auth.password –v {PASSWORD}
iscsiadm -m node -T {IQN} –p {SERVER} -o update -n node.startup -v automatic
```

This data is saved and will be used to connect to logical disk {IQN}. To bind to the target just now use the command:

iscsiadm -m node –login -T {IQN} –p {SERVER}

Targets configured with the provided attributes are stored inside the **/etc/iscsi/nodes** folder persistently, configuration files can be manually edited. In the example above it was set **node.startup = automatic**, when the machine starts the connection to this target (login) will be automatically.

iSCSI Initiator – Linux

For each logical disk to which the system connects (login to target), a new disk appears in the system, for example if the system only had disk **/dev/sda** a new disk /dev/sbd will appear.

The operating system treats this new disk as if it were a local physical disk, although in reality it is an iSCSI logical disk. If it is a newly created disk (no target) the **fdisk** command should be used to define the partitions that can then be formatted with the **mkfs** command. Being formatted they can then be mounted and eventually registered in **/etc/fstab** to be mounted whenever the system boots.

Assigned local disk identification can be a problem when using multiple iSCSI logical disks. Assigning local disks is in order of binding, consistently maintaining mappings between local disk identifiers and targets requires some more work. For example, if a target is not available, the local disk identifiers for the other **targets** may differ.

An advantage of using a standard protocol like iSCSI is interoperability, for example Windows *targets* can be used by Linux *initiators* and Linux *targets* can be used by Windows *initiators*.